

Nghiên cứu và xây dựng hệ thống khôi phục dấu câu tiếng Việt

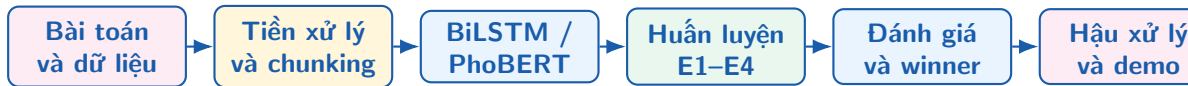
So sánh BiLSTM và PhoBERT trong pipeline tích hợp Whisper ASR

Nguyễn Quang Sang Đỗ Khắc Phúc Thịnh Trần Bùi Hà Giang

Viện Trí tuệ Nhân tạo – Trường Đại học Công nghệ, ĐHQGHN

Tháng 08 năm 2026

Mạch trình bày: đi theo đúng luồng xử lý của hệ thống



Thông điệp xuyên suốt

Mỗi quyết định ở bước trước tạo ra đầu vào và ràng buộc cho bước sau: cách tạo nhãn ảnh hưởng đến alignment; mất cân bằng nhãn ảnh hưởng đến loss; metric trên validation quyết định winner; lỗi của winner định hướng hậu xử lý và phát triển tiếp theo.

Đặt vấn đề: ASR nhận đúng từ nhưng văn bản vẫn khó đọc

Đầu ra ASR thô

bạn đã hoàn thành bài tập chưa ngày
mai chúng ta nộp bài

Ba hệ quả

- Người đọc không thấy ranh giới câu.
- Dấu hỏi, liệt kê và quan hệ giữa các vế bị mất.
- Dịch máy, tóm tắt, trích xuất thông tin bị giảm chất lượng.

Âm thanh tiếng Việt



Whisper: chuỗi từ thô



Cần khôi phục cấu trúc câu

Mục tiêu và phạm vi của bài tập lớn

- ❶ Khôi phục ba dấu câu: **phẩy, chấm, hỏi**.
- ❷ Phát biểu bài toán dưới dạng **gán nhãn chuỗi theo từ**.
- ❸ So sánh **BiLSTM baseline** với **PhoBERT** và hai chế độ class weight.
- ❹ Chọn winner bằng Punctuation Macro-F1 trên validation.
- ❺ Đóng gói pipeline với Whisper, hậu xử lý và Streamlit.

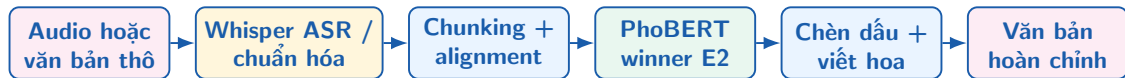
Phạm vi đánh giá

Đánh giá định lượng chính thực hiện trên văn bản sạch JointCapPunc.

Chưa đánh giá

Chưa đo end-to-end từ audio qua Whisper đến dấu câu cuối cùng; vì vậy chưa lượng hóa tác động của lỗi ASR lên mô hình.

Luồng hoạt động end-to-end của hệ thống



Cơ chế tổng quát

Whisper hoặc người dùng cung cấp chuỗi từ không dấu câu. Chuỗi được chuẩn hóa và chia đoạn, PhoBERT dự đoán một nhãn sau mỗi từ, sau đó module hậu xử lý biến nhãn thành ký tự dấu câu, sửa khoảng trắng và viết hoa đầu câu.

Ví dụ dữ liệu thật: một lượt chạy từ đầu vào đến đầu ra

Input

bạn đã hoàn thành bài tập chưa
ngày mai chúng ta nộp bài

dự đoán nhãn



Output

**Bạn đã hoàn thành bài tập chưa?
Ngày mai chúng ta nộp bài.**

- “chưa” → QUESTION
- “bài” cuối → PERIOD
- Từ sau QUESTION được viết hoa

Mô hình không sinh lại câu; nó chỉ quyết định dấu câu nào xuất hiện sau từng từ.

Bước 1 – Phát biểu bài toán gán nhãn chuỗi

Cho chuỗi từ không có dấu câu:

$$X = (x_1, x_2, \dots, x_N)$$

Mô hình dự đoán chuỗi nhãn có cùng độ dài:

$$Y = (y_1, y_2, \dots, y_N), \quad y_i \in \{O, \text{COMMA}, \text{PERIOD}, \text{QUESTION}\}$$

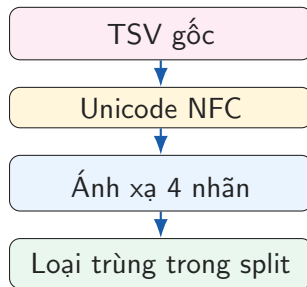
Từ	bạn	xong	chưa	ngày	mai
Nhãn	O	O	QUESTION	O	PERIOD

Lợi ích của cách biểu diễn

Bài toán trở thành phân loại 4 lớp tại từng vị trí, cho phép dùng BiLSTM hoặc Transformer Token Classification và tối ưu trực tiếp bằng Cross-Entropy Loss.

Bước 2 – Chuẩn hóa dữ liệu JointCapPunc

- 1 Đọc TSV gồm token, nhãn viết hoa và nhãn dấu câu.
- 2 Chuẩn hóa Unicode tiếng Việt về dạng **NFC**.
- 3 Bảo toàn dấu thanh và từ phức.
- 4 Ánh xạ QMARK → QUESTION.
- 5 Loại exact normalized duplicates trong từng split.



Điểm cần bổ sung

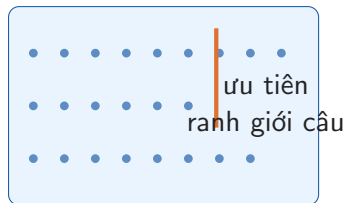
Báo cáo chưa xác nhận duplicate giữa Train–Validation–Test; đây là kiểm tra cần có để loại trừ rò rỉ dữ liệu giữa các split.

Bước 3 – Chunking: giữ ngữ cảnh nhưng không vượt giới hạn mô hình

Thuật toán chunking

- 1 Gom tối đa **150 lexical words**.
- 2 Ưu tiên cắt tại PERIOD hoặc QUESTION gần nhất.
- 3 Nếu không có ranh giới tự nhiên, cắt tại giới hạn.
- 4 Giữ đồng bộ chuỗi từ và chuỗi nhãn.

Chuỗi dài → các chunk



Rủi ro kỹ thuật

150 words không đảm bảo dưới 192 subwords của PhoBERT. Cần thống kê tỷ lệ truncate sau tokenization để biết có mất nhãn hoặc ngữ cảnh hay không.

Bước 4 – Last-subword alignment cho PhoBERT

PhoBERT dùng BPE nên một từ có thể tách thành nhiều subword. Nhãn đầu câu thuộc về **toàn bộ từ**, vì vậy dự án gán nhãn cho subword cuối:

Ví dụ alignment

Từ: hoàn_thành

Subwords: hoàn | thành

Nhãn loss: -100 | PERIOD

Cơ chế

- Subword đầu/giữa: nhãn -100.
- Subword cuối: nhận nhãn thật của từ.
- Cross-Entropy bỏ qua mọi vị trí -100.

Tại sao cần?

Nếu gán cùng một nhãn cho mọi subword, từ dài sẽ đóng góp loss nhiều lần và làm lệch phân bố huấn luyện.

Dữ liệu thật sau tiền xử lý: gần 7,9 triệu token

Split	Samples	Tokens	O	COMMA	PERIOD	QUESTION
Train	38,066	4,940,478	91.10%	4.62%	3.73%	0.56%
Validation	15,227	1,977,406	91.06%	4.64%	3.74%	0.56%
Test	22,832	2,968,815	91.04%	4.64%	3.76%	0.56%

38,066

train samples

4.94M

train tokens

0.56%

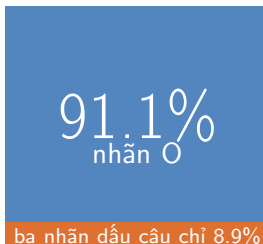
QUESTION

Ba split có phân bố gần như giống nhau, nhưng mất cân bằng lớp rất mạnh.

Từ mất cân bằng lớp đến lựa chọn metric

Vì sao Accuracy dễ gây hiểu nhầm?

Nếu luôn dự đoán nhãn O, mô hình đã đạt khoảng 91% accuracy nhưng không khôi phục được một dấu câu nào.



Metric chính

$$\text{Punct-Macro-F1} = \frac{F1_{comma} + F1_{period} + F1_{question}}{3}$$

Không tính lớp O.

Mô hình cơ sở E1 – BiLSTM nhìn ngữ cảnh hai chiều



Cơ chế cốt lõi

LSTM thuận đọc ngữ cảnh bên trái; LSTM nghịch đọc ngữ cảnh bên phải. Hai trạng thái được nối để dự đoán nhãn cho mỗi từ.

Vai trò trong nghiên cứu

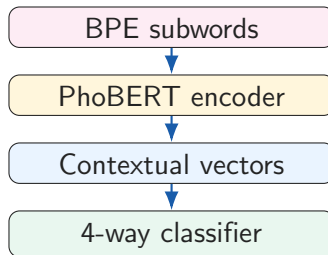
E1 là baseline nhỏ gọn (16.9 MB), cho biết mức hiệu năng khi huấn luyện từ đầu mà không dùng mô hình ngôn ngữ tiền huấn luyện.

Mô hình chính – PhoBERT Token Classification

Cơ chế Transformer

Self-attention cho phép mỗi subword tham chiếu trực tiếp tới toàn bộ ngữ cảnh trong chunk. Biểu diễn đã tiền huấn luyện trên tiếng Việt giúp mô hình hiểu cấu trúc câu và quan hệ ngữ nghĩa tốt hơn.

- Backbone: **PhoBERT-base-v2**
- Đầu ra: linear classifier cho 4 nhãn
- Giới hạn: 192 subwords
- Kích thước checkpoint: 514.7 MB



Bốn thí nghiệm E1–E4: thay kiến trúc và chiến lược loss

Mã	Kiến trúc	Class weight	Mục đích
E1	BiLSTM	Không	Baseline huấn luyện từ đầu
E2	PhoBERT	Không	Tận dụng pretraining
E3	PhoBERT	Full inverse	Nhấn mạnh mạnh lớp hiếm
E4	PhoBERT	Sqrt inverse	Nhấn mạnh vừa phải lớp hiếm

Giả thuyết thực nghiệm

Vì QUESTION chỉ chiếm 0.56%, class weighting có thể cải thiện recall lớp hiếm. Tuy nhiên nếu trọng số quá lớn, mô hình có thể sinh quá nhiều dấu câu và làm giảm precision.

Thuật toán class weighting trong E3 và E4

E3 – Full inverse

$$w_c = \frac{N_{total}}{K N_c}$$

Lớp càng hiếm thì trọng số càng lớn; lỗi trên QUESTION được phạt mạnh.

E4 – Sqrt inverse

$$w_c = \sqrt{\frac{N_{total}}{K N_c}}$$

Căn bậc hai làm dịu chênh lệch, giảm nguy cơ mô hình over-predict lớp hiếm.

Cách loss hoạt động

Cross-Entropy tại mỗi vị trí được nhân với w_c của nhãn thật. Alignment –100 vẫn được bỏ qua, sau đó loss trung bình được backpropagate qua classifier và PhoBERT.

Thiết lập huấn luyện: kiểm soát để so sánh công bằng

Siêu tham số	E1	E2	E3	E4
Max length	150 words	192 subwords	192 subwords	192 subwords
Batch hiệu dụng	128	16	16	16
Optimizer	AdamW	AdamW	AdamW	AdamW
Learning rate	10^{-3}	3×10^{-5}	3×10^{-5}	3×10^{-5}
Weight decay	10^{-4}	0.01	0.01	0.01
Warmup ratio	–	0.10	0.10	0.10
Epoch	12	5	5	5

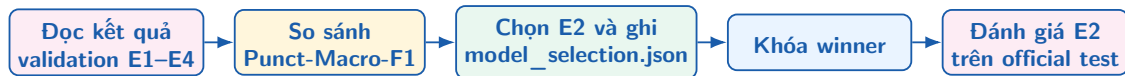
Checkpointing

Sau mỗi epoch, đánh giá trên validation; chỉ giữ checkpoint có Punctuation Macro-F1 tốt nhất trong thư mục tương ứng E1–E4.

Diễn giải thận trọng

E2–E4 đều tốt nhất ở epoch 5, nhưng đây cũng là epoch cuối; chưa đủ bằng chứng rằng mô hình đã hội tụ.

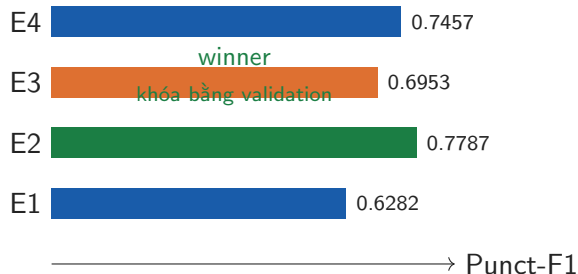
Quy trình chọn winner: khóa mô hình trước khi xem test



Ý nghĩa phương pháp luận

Test không tham gia lựa chọn mô hình. Kết quả của E1, E3, E4 trên test chỉ là post-hoc analysis, không được dùng để thay đổi winner sau khi đã quan sát test.

Kết quả validation: E2 được chọn làm winner



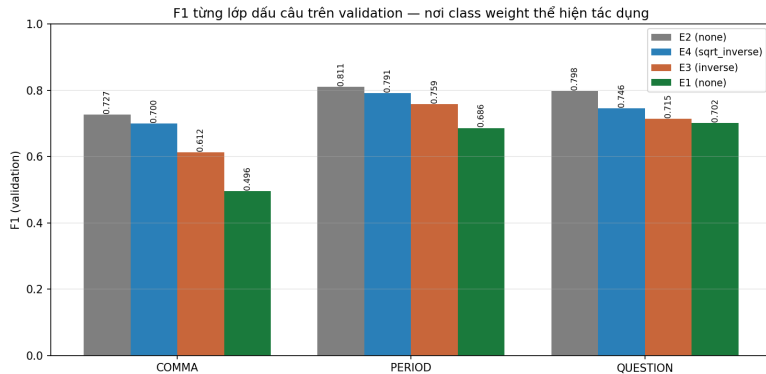
Validation Punct-F1

- **E2: 0.7787**
- E4: 0.7457
- E3: 0.6953
- E1: 0.6282

Kết luận

PhoBERT không class weight đạt kết quả tốt nhất; tăng trọng số lớp hiếm không giúp trong các cấu hình đã thử.

Vì sao class weighting không thắng? Nhìn theo từng lớp



Quan sát

E2 dẫn đầu ở cả ba dấu câu; full inverse giảm mạnh COMMA, còn sqrt-inverse vẫn thấp hơn E2.

Official Test của winner E2: kết quả ổn định

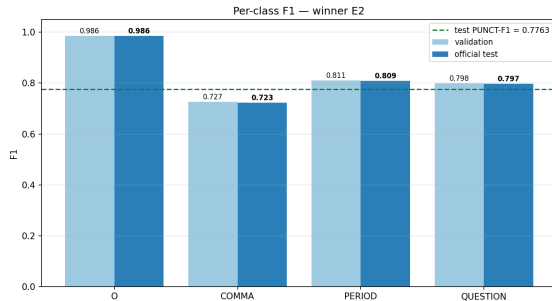
0.7763

Punctuation Macro-F1

0.9663

Overall Accuracy

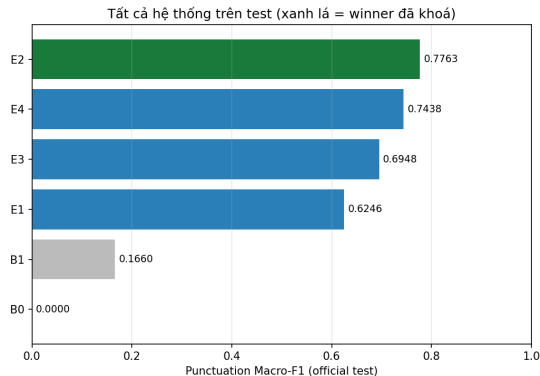
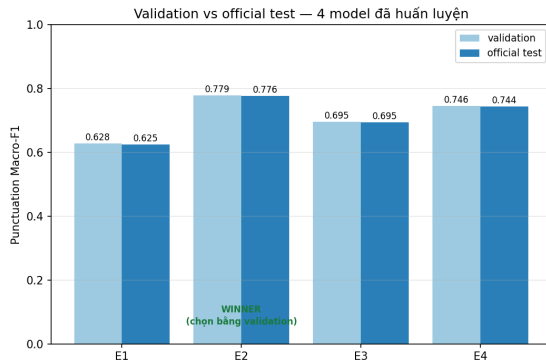
Nhãn	Precision	Recall	F1
COMMA	0.7407	0.7054	0.7226
PERIOD	0.7948	0.8240	0.8091
QUESTION	0.7484	0.8528	0.7972



Thông điệp

Validation và test gần nhau ở mọi lớp; COMMA vẫn là lớp khó nhất do ranh giới vế và liệt kê mơ hồ.

So sánh post-hoc: lợi ích thực tế của PhoBERT

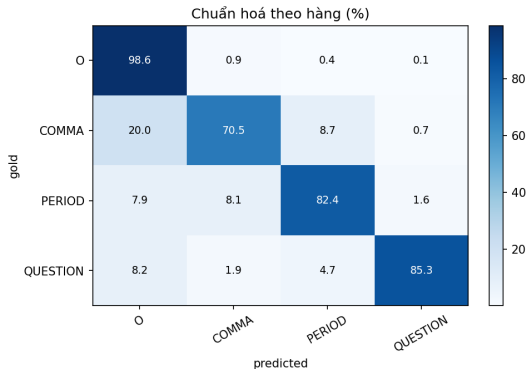
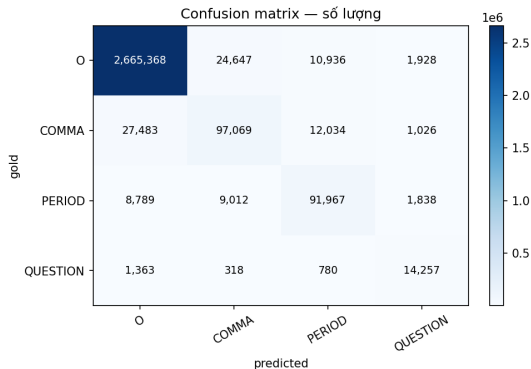


Kết quả chính

E2 đạt 0.7763, cao hơn E1 BiLSTM 0.6246. Tuy nhiên chênh lệch này phản ánh đồng thời kiến trúc Transformer và lợi ích pretraining; thí nghiệm hiện tại chưa tách riêng hai nguyên nhân.

Ma trận nhầm lẫn: mô hình đang sai ở đâu?

Official test confusion matrix — winner E2



- COMMA đúng 70.5%, nhưng 20.0% bị bỏ thành O và 8.7% bị nâng thành PERIOD.
- PERIOD đúng 82.4%; QUESTION đúng 85.3%.
- Lỗi quan trọng nhất không phải dấu hỏi, mà là xác định ranh giới mềm của dấu phẩy.

Từ lỗi mô hình đến hậu xử lý và demo

Ba lỗi điển hình từ dữ liệu thật

- COMMA → O: bỏ sót ngắt ở chuỗi liệt kê.
- QUESTION → PERIOD: câu hỏi gián tiếp thiếu tín hiệu rõ.
- O → QUESTION: đặt dấu hỏi quá sớm sau cụm “xin hỏi”.

Post-processing pipeline

- 1 Chèn dấu liền sau từ.
- 2 Chuẩn hóa khoảng trắng.
- 3 Viết hoa đầu văn bản và sau chấm/hỏi.
- 4 Bảo toàn URL, email, số thập phân.

Đóng gói ứng dụng

Streamlit tự load winner từ `model_selection.json`; người dùng nhập văn bản thô hoặc chạy Whisper, sau đó nhận văn bản đã khôi phục dấu câu và viết hoa.

Kết luận và hướng phát triển

Những gì hệ thống đã đạt được

- Hoàn chỉnh pipeline dữ liệu → mô hình → hậu xử lý → demo.
- E2 đạt Punctuation Macro-F1 = **0.7763** trên 2,968,815 token test.
- PhoBERT vượt BiLSTM baseline trong thiết lập thực nghiệm.
- Kết quả validation và test nhất quán.

Bốn bước tiếp theo

- 1 Đánh giá audio end-to-end với WER.
- 2 Joint capitalization + punctuation.
- 3 Kiểm tra truncate sau BPE.
- 4 Kiểm tra duplicate giữa các split.

Cảm ơn thầy và các bạn!

Q&A